
多智能体运维诊断协作系统

基于大模型的多智能体协作框架

—— 应用于运维故障诊断场景

毕业设计 · 项目方案说明书

王 志 海
2026 年 7 月

一 项目概述

1.1 背景与问题

现代运维体系中，一次故障排查通常涉及多个数据源：服务器指标（CPU、内存）、数据库状态（慢 SQL、锁等待）、日志文件（错误异常）。工程师需要在多个平台间手动切换、关联信息才能定位根因，效率低、门槛高。

本项目的目标：让多个 AI Agent 自动完成「全面排查 → 辩论共识 → 反思复审 → 输出报告」的完整闭环，将运维诊断从「人工翻查」升级为「智能协作」。

1.2 项目定位

毕业设计方向	大模型与多智能体
核心贡献	多 Agent 协作模式（直达/链式/并行）的设计与对比实验
项目类型	框架 + 应用，以运维诊断为落地场景
与 Java 版区别	Java 版是单 Agent + 多工具；本毕设聚焦多 Agent 协作模式研究

二 技术栈

主语言	Python 3.10+ — 大模型生态丰富，快速迭代
Agent 编排	LangGraph — 原生支持多 Agent 图编排
LLM 部署	Ollama（本地运行）— 无需外网 API，演示稳定
API 层	FastAPI — Agent 能力暴露 + 前端对接
前端	React 18 + TypeScript + Ant Design + ECharts
数据源	psutil（服务器实时指标）+ MySQL 真实连接 + 日志文件

三 系统架构 — 四层协作模型

系统采用四层架构：Coordinator 动态路由 → 领域 Agent 执行诊断 → 质量保障机制 (Debate + Reflection) → Report Agent 输出报告。

3.1 Coordinator — 三种路由策略

Coordinator 作为「总指挥」，由 LLM 根据用户问题动态决策路由策略：

直达策略

问题明确指向某个领域 → 直接路由目标 Agent

例："这个 SQL 为什么慢？" → DB Agent

链式策略

问题模糊 → 逐层推理追查根因

例："系统很慢" → Server Agent → DB Agent → Log Agent

并行策略

需要全面检查 → 同时分发到多个 Agent

例："大促结束，全链路体检"

3.2 领域 Agent

每个 Agent 拥有独立工具集，专精一个诊断维度：

Server Agent — 服务器诊断

check_cpu() / check_memory() / check_disk() / check_process() / check_thread() / check_network()

数据源：psutil

DB Agent — 数据库诊断

EXPLAIN 分析 / 索引检测 / SHOW PROCESSLIST / 连接池监控 / 慢查询终止（高急需审批） 数据源：MySQL

Log Agent — 日志分析

关键字检索 / 错误聚合 / 异常模式发现 / 慢查询日志分析 数据源：日志文件 + MySQL slow_log

四 质量保障机制

4.1 Debate Arena（辩论场 — 可选触发）

当并行模式下多个 Agent 诊断结论不一致时触发。各 Agent 提供证据辩护，Coordinator 或投票机制裁决，最终达成共识。

设计意义：当多个专家意见不一时，单一 Agent 结论不可信。通过辩论让证据说话，提升诊断可信度。

4.2 Reflection（反思复审 — 必备流程）

每份诊断报告生成后，其他 Agent 交叉审核对应部分：DB Agent 审核数据库证据、Server Agent 审核服务器证据、Log Agent 审核日志是否支持结论。有遗漏则退回修改。

设计意义：单一 Agent 可能有盲区或误判，交叉复审相当于「提交前 Review 代码」，是质量保证的最后一道防线。

五 完整协作流程示例

例：直达模式 — 「这个 SQL 为什么慢？」

用户：帮我分析这个 SQL：SELECT * FROM orders WHERE status = 'PENDING'

|

Coordinator：路由策略 = 直达，目标 = DB Agent

|

DB Agent：

① explain_sql → type=ALL 全表扫描

② show_create_table → status 字段无索引

③ 综合判断：缺索引导致全表扫描

|

Reflection → 审核通过 → 输出报告

例：链式模式 — 「系统很慢，经常超时」

Step 1 — Server Agent：CPU 80%，MySQL 进程占 50%
→ 初步判断：MySQL 负载高是瓶颈

Step 2 — DB Agent：大量 SELECT WHERE status='PENDING'
→ explain 全表扫描 50000 行，status 字段缺索引

Step 3 — Log Agent：大量连接超时日志，慢查询反复出现
→ 判断：慢 SQL 导致了连锁反应

Final → Reflection → Report（缺索引→全表扫描→CPU飙升→超时）

例：并行模式 — 「检查系统整体健康度」

用户：明天大促，帮我看看整体健康度

|

Coordinator：并行分发到所有 Agent

|

Server Agent → CPU/内存/磁盘 全部正常

DB Agent → 有慢查询风险（status 缺索引）

Log Agent → 无明显异常

|

结论一致 → 无需 Debate → Reflection → Report

建议：系统整体健康，上线前给 status 加索引

六 实施计划

阶段	内容	交付物	时间
P1	LangGraph 多 Agent 框架搭建	多 Agent 原型可运行	2 周
P2	DB Agent — MySQL 真实连接	DB Agent 可诊断	1 周
P3	Server Agent — psutil 采集	Server Agent 可用	1 周
P4	Log Agent — 日志解析	Log Agent 可用	1 周
P5	Debate + Reflection 机制	质量保障层跑通	2 周
P6	Report Agent + 结构化报告	报告引擎完成	1 周

P7	前端可视化 React + TS	前端可展示	2 周
P8	复合测试 + 对比实验	实验数据 + 分析	2 周
P9	论文撰写 + 答辩准备	论文定稿	3 周

七 论文结构建议

摘要

第一章 绪论

第二章 相关技术

第三章 系统设计

第四章 系统实现

第五章 实验与评估

第六章 总结与展望

研究背景 / 国内外现状 / 本文工作
LLM + Function Calling / 多智能体协作 / LangGraph / 运维诊断
四层架构 / 动态路由 / 领域 Agent / 质量保障 / 报告生成
Agent 框架 / 各领域 Agent / 协作机制 / 前端可视化
三种协作模式对比 / Debate 有效性 / Reflection 影响分析

八 项目创新点与亮点

多 Agent 协作模式研究

设计并实现三种路由策略（直达/链式/并行），通过对比实验验证各模式在不同场景下的效果差异，而非单一功能实现。

Debate + Reflection 双保险

引入多 Agent 辩论机制解决意见分歧，Reflection 交叉复审确保报告质量。这两层机制在现有运维诊断系统中较为鲜见。

真实数据源驱动

使用 psutil 实时采集服务器指标、真实 MySQL 连接、本地日志文件作为数据源，避免 Mock 数据，诊断结果真实可信。

完整闭环体验

从「用户提问」到「诊断报告输出」形成完整闭环，前端可视化展示诊断链路 with 指标图表，直观易用。

九 待确认事项（与导师讨论）

- 论文题目方向是否合适
- Agent 数量：DB + Server + Log 起步是否够用
- 是否需要扩展到 Redis / MQ Agent
- Debate 机制的具体形式（投票制 / Coordinator 裁决 / 其他）
- 实验对比维度（准确率 / 耗时 / 用户满意度）
- 时间节点和里程碑要求

—— 谢谢！ ——